Test Report

Blockchain-Based Proof-of-Attendance Protocol (POAP) System for Event Attendance Verification

1. Introduction

This test report documents the testing activities conducted for the Blockchain-Based Proof-of-Attendance Protocol (POAP) System for Event Attendance Verification. The purpose of testing is to verify that the smart contract functions operate correctly, enforce access control, reject invalid inputs, and maintain reliable badge issuance records.

The system was implemented using Solidity and tested using the Hardhat development framework with Mocha and Chai. The main smart contract tested in this project is AttendanceBadges.sol, which implements an ERC-1155-based attendance badge system. The test cases cover deployment, event creation, POAP badge issuance, batch issuance, issuer role management, validation checks, revert conditions, gas usage, and code coverage.

2. Testing Environment

Item	Description
Smart Contract	AttendanceBadges.sol
Blockchain Standard	ERC-1155
Programming Language	Solidity
Development Framework	Hardhat
Testing Framework	Mocha and Chai
Network Used for Testing	Hardhat Local Network
Compiler Version	Solidity 0.8.28
Test Command	npm run test or npx hardhat test
Gas Test Command	npm run test:gas
Coverage Command	npm run test:coverage

3. Testing Scope

The testing process focused on the following smart contract functions and features:

1. Contract deployment and constructor validation.

2. POAP event badge creation.
3. Single badge issuance.
4. Batch badge issuance.
5. Issuer role granting and revocation.
6. Access control enforcement.
7. Input validation and revert conditions.
8. Maximum supply enforcement.
9. Event active/inactive status update.
10. Read functions for attendee badge verification.
11. Gas usage measurement.
12. Code coverage analysis.

4. Functional Test Summary

The automated test suite was executed using Hardhat. A total of 14 automated test groups passed successfully. The test result confirms that the core smart contract functions behaved according to the expected requirements.

Test Category	No. of Test Groups	Result
Deployment	2	Passed
Event Creation	3	Passed
Badge Issuance	6	Passed
Admin Roles and Read Validation	3	Passed
Total	14	Passed

Overall Result: All automated tests passed successfully.

5. Detailed Test Case Table

No.	Function / Area Tested	Input / Condition	Expected Output	Actual Output	Status
1	Constructor	Valid owner and default URI	Contract deploys successfully and owner is initialized	Owner, roles, ERC-1155 interface, and next token ID verified	Pass

2	Constructor	Zero owner address	Transaction should revert	Reverted with owner validation error	Pass
3	Constructor	Empty default URI	Transaction should revert	Reverted with default URI validation error	Pass
4	createPOAPEvent	Valid metadata URI, max supply, and event time window	New POAP event token ID is created	Event emitted and metadata readable	Pass
5	createPOAPEvent	Empty token URI	Transaction should revert	Reverted with token URI validation error	Pass
6	createPOAPEvent	Zero max supply	Transaction should revert	Reverted with max supply validation error	Pass
7	createPOAPEvent	Invalid event time window	Transaction should revert	Reverted with invalid event window error	Pass
8	createPOAPEvent	Non-owner caller	Transaction should revert	Reverted due to owner-only restriction	Pass
9	setPOAPEventActive	Owner updates event status	Event active status should change	Status event emitted and active flag updated	Pass

10	setPOAPEventActive	Non-owner caller	Transaction should revert	Reverted due to owner-only restriction	Pass
11	setPOAPEventActive	Missing token ID	Transaction should revert	Reverted because token does not exist	Pass
12	issuePOAP	Owner issues badge to attendee	Badge should be minted and attendee balance becomes 1	POAP issued and balance verified	Pass
13	issuePOAP	Authorized issuer issues badge	Badge should be minted successfully	Badge issued and balance updated	Pass
14	issuePOAP	Unauthorized caller	Transaction should revert	Reverted because caller is not issuer	Pass
15	issuePOAP	Zero attendee address	Transaction should revert	Reverted with zero address validation error	Pass
16	issuePOAP	Missing token ID	Transaction should revert	Reverted because token does not exist	Pass
17	issuePOAP	Duplicate attendee and token	Transaction should revert	Reverted because attendee already issued	Pass
18	issuePOAP	Inactive event	Transaction should revert	Reverted because event is inactive	Pass

19	issuePOAP	Max supply exceeded	Transaction should revert	Reverted because max supply reached	Pass
20	batchIssuePOAPs	Three valid attendees	All attendees should receive badge	Batch balances and minted count verified	Pass
21	batchIssuePOAPs	Empty attendee array	Transaction should revert	Reverted because no attendees provided	Pass
22	batchIssuePOAPs	Zero attendee address	Transaction should revert	Reverted with zero address validation error	Pass
23	batchIssuePOAPs	Batch size exceeds remaining supply	Transaction should revert	Reverted because max supply reached	Pass
24	grantIssuer	Valid issuer address	Issuer role should be granted	hasRole returned true	Pass
25	grantIssuer	Zero address	Transaction should revert	Reverted with issuer zero address validation error	Pass
26	revokeIssuer	Valid issuer address	Issuer role should be revoked	hasRole returned false	Pass

27	revokeIssuer	Zero address	Transaction should revert	Reverted with issuer zero address validation error	Pass
28	getPOAPEvent	Missing token ID	Transaction should revert	Reverted because token does not exist	Pass
29	uri	Missing token ID	Transaction should revert	Reverted because token does not exist	Pass
30	hasPOAP	Zero attendee address	Transaction should revert	Reverted with zero address validation error	Pass
31	getHolderTokenIds	Zero attendee address	Transaction should revert	Reverted with zero address validation error	Pass
32	getHolderPOAPBalances	Zero attendee address	Transaction should revert	Reverted with zero address validation error	Pass

6. Console Test Output (ss for apdx)

The automated test suite was executed using the following command:

npm run test

The test output showed:

AttendanceBadges
  deployment
    ✔ initializes owner, issuer role, ERC-1155 interface, and next token ID
    ✔ rejects invalid constructor arguments
  event creation

✔ creates a badge type and exposes its metadata
✔ rejects invalid event creation input
✔ restricts event creation and status changes to the owner
issuing badges
✔ allows the owner to issue a POAP and updates attendee reads
✔ allows an owner-granted issuer to issue badges
✔ batch issues one POAP to multiple attendees
✔ rejects invalid issue attempts
✔ enforces max supply for single and batch issue flows
✔ rejects invalid batch issue input
admin roles and read validation
✔ grants and revokes issuer permissions
✔ rejects invalid role and read inputs
✔ emits status changes and rejects missing token status updates

14 passing

The result confirms that all automated tests passed successfully.

7. Gas Consumption Analysis

Gas usage analysis was conducted to measure the transaction cost of important smart contract operations. The following command was used:

npm run test:gas

The gas result is summarized below:

Function	Minimum Gas	Average Gas	Maximum Gas	Calls
Deployment	2,563,405	2,563,405	2,563,405	14
createPOAPEvent	167,228	170,000	194,949	10
issuePOAP	179,001	179,584	181,333	4
batchIssuePOAPs	381,015	381,015	381,015	1
grantIssuer	48,691	48,691	48,691	2
revokeIssuer	26,651	26,651	26,651	1
setPOAPEventActive	27,803	27,803	27,803	2
getPOAPEvent	28,576	28,576	28,576	4
getHolderPOAPBalances	29,831	29,831	29,831	1
balanceOfBatch	33,096	33,096	33,096	1

Based on the gas results, batchIssuePOAPs consumed more gas than single issuance because it issues badges to multiple attendees in one transaction. However, it is still more practical for large events because

it reduces repetitive manual transactions and allows multiple attendees to receive POAP badges in one operation.

8. Code Coverage Result

Code coverage testing was performed to measure how much of the smart contract source code was executed during testing. The following command was used:

npm run test:coverage

The coverage result is summarized below:

Contract File	Line Coverage	Statement Coverage	Uncovered Lines
contracts/AttendanceBadges.sol	100.00%	100.00%	None
Total	100.00%	100.00%	None

The result shows that the automated test suite covered the full smart contract logic. This improves confidence that the main functions, validation controls, and revert conditions were tested properly.

9. Failed Test Explanation

No failed tests were recorded during the final test execution. All 14 automated test groups passed successfully.

Some test cases were intentionally designed to trigger transaction reverts. These include unauthorized access, zero address input, empty metadata URI, invalid event window, duplicate issuance, missing token ID, inactive event, and maximum supply overflow. These are not actual system failures. Instead, they prove that the smart contract correctly rejects invalid or unauthorized operations.

10. Security Validation Summary

The testing process confirmed that the smart contract implemented several important security and validation controls:

Security / Validation Area	Evidence from Testing
Owner-only access control	Non-owner event creation and status update attempts reverted
Issuer role control	Unauthorized badge issuance attempts reverted
Zero address validation	Zero attendee and issuer addresses rejected
Duplicate issuance prevention	Same attendee could not receive the same badge twice

Maximum supply enforcement	Issuance stopped when max supply was reached
Event status enforcement	Badge issuance rejected when event was inactive
Missing token validation	Read and update operations rejected invalid token IDs
ERC-1155 balance verification	Badge ownership verified using balance checks

11. Integration / Frontend Testing

The frontend was tested to verify that users could interact with the smart contract through the web interface.

Test Area	Expected Result	Actual Result	Status
MetaMask connection	Wallet connects and address is displayed	Wallet connects with the address displayed	Pass
Network check	UI detects correct or wrong network	UI detects correct network	Pass
Read function	User can view badge or event data	User can view badge and event data	Pass
Write function	User can create event or issue badge through MetaMask	User can create event and issue badges	Pass
Transaction status	UI shows pending, confirmed, or failed transaction	UI shows pending and confirmed transactions	Pass
Error handling	UI displays readable error message	Readable error message	Pass

12. Deployment Testing

Deployment testing was conducted to confirm that the smart contract could be deployed to the selected blockchain environment.

Deployment Item	Details
Deployment Network	Sepolia
Deployment Tool	Hardhat
Contract Name	AttendanceBadges
Contract Address	0x2d0c280dfeff1e258231380df6288ff867a2ec55

Transaction Hash	0xfbbc0f45308d8a40d59c9dac294acd59f0becf661fb4439b11d4eee429d89f10
Etherscan Link	https://sepolia.etherscan.io/address/0x2d0c280dfeff1e258231380df6288ff867a2ec55
ABI File Location	deployments/sepolia/AttendanceBadges.abi.json
Deployment Script	scripts/deploy-attendance-badges.ts

The deployment result confirms that the smart contract can be deployed and interacted with using the selected blockchain environment.

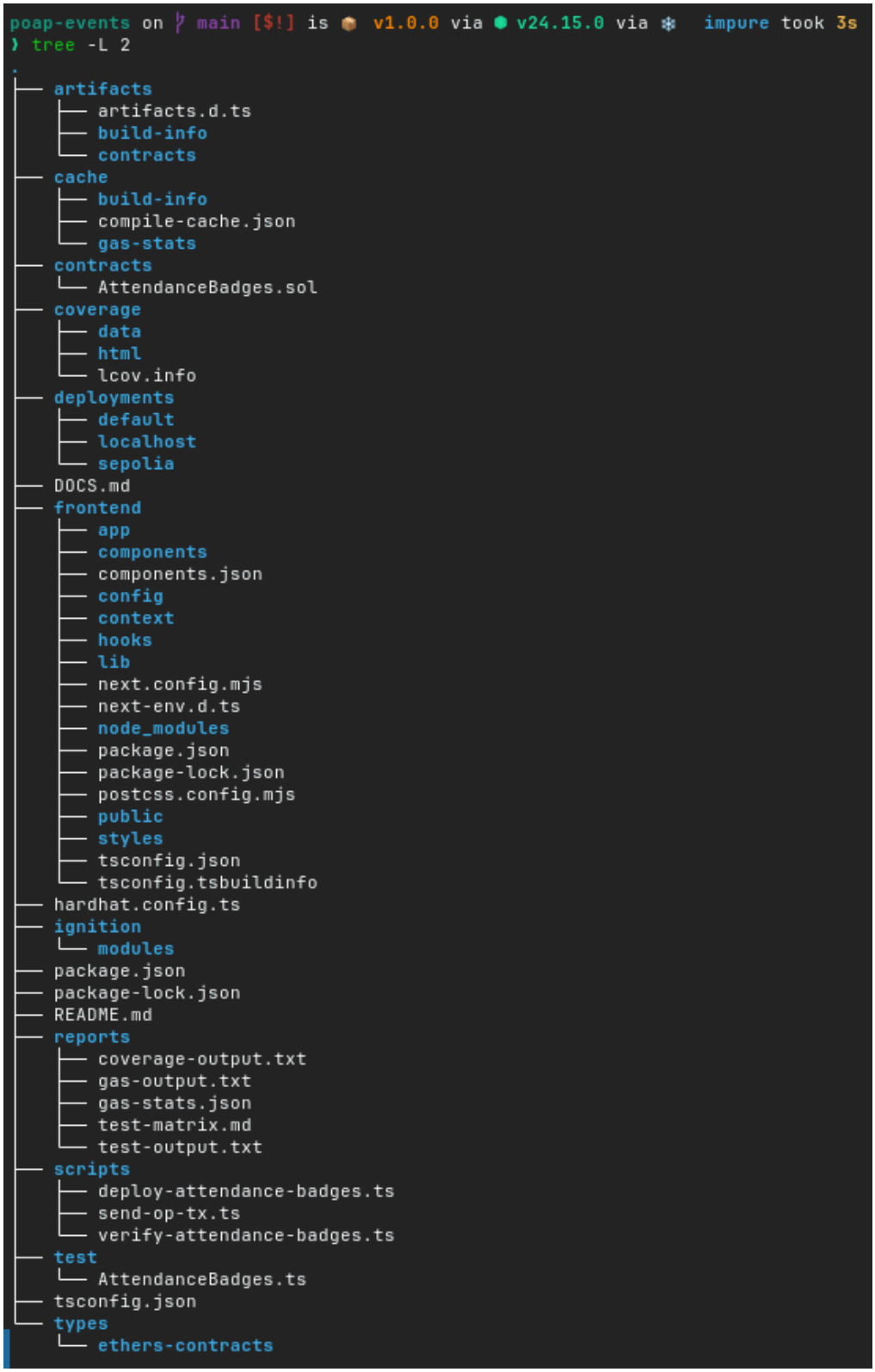
13. Conclusion

The testing process confirmed that the Blockchain-Based POAP Attendance System operated correctly according to the expected requirements. The smart contract successfully handled event creation, single badge issuance, batch badge issuance, issuer role management, event status updates, and attendance badge verification.

All 14 automated test groups passed successfully. The test suite also covered access control, edge cases, revert conditions, and validation checks. Gas usage analysis showed that the contract functions consumed measurable and acceptable gas costs, while batch issuance provided a more practical method for issuing badges to multiple attendees. Code coverage analysis showed 100% line and statement coverage for the main smart contract.

Overall, the test results demonstrate that the proposed smart contract is functional, properly validated, and suitable for supporting blockchain-based proof-of-attendance verification.

14. Appendix (screenshots)


poap-events on main [$!] is v1.0.0 via v24.15.0 via impure took 3s
❯ tree -L 2
.
├── artifacts
│   ├── artifacts.d.ts
│   ├── build-info
│   └── contracts
├── cache
│   ├── build-info
│   ├── compile-cache.json
│   └── gas-stats
├── contracts
│   └── AttendanceBadges.sol
├── coverage
│   ├── data
│   ├── html
│   └── lcov.info
├── deployments
│   ├── default
│   ├── localhost
│   └── sepolia
├── DOCS.md
├── frontend
│   ├── app
│   ├── components
│   ├── components.json
│   ├── config
│   ├── context
│   ├── hooks
│   ├── lib
│   ├── next.config.mjs
│   ├── next-env.d.ts
│   ├── node_modules
│   ├── package.json
│   ├── package-lock.json
│   ├── postcss.config.mjs
│   ├── public
│   ├── styles
│   ├── tsconfig.json
│   └── tsconfig.tsbuildinfo
├── hardhat.config.ts
├── ignition
│   └── modules
├── package.json
├── package-lock.json
├── README.md
├── reports
│   ├── coverage-output.txt
│   ├── gas-output.txt
│   ├── gas-stats.json
│   ├── test-matrix.md
│   └── test-output.txt
├── scripts
│   ├── deploy-attendance-badges.ts
│   ├── send-op-tx.ts
│   └── verify-attendance-badges.ts
├── test
│   └── AttendanceBadges.ts
├── tsconfig.json
└── types
    └── ethers-contracts

File: contracts/AttendanceBadges.sol

```solidity
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.28;

import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";
import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
import {ERC1155} from "@openzeppelin/contracts/token/ERC1155/ERC1155.sol";
import {ERC1155Supply} from "@openzeppelin/contracts/token/ERC1155/extensions/ERC1155Supply.sol";

/// @title POAP Events
/// @author POAP Events Prototype
/// @notice Issues event-attendance badges as ERC-1155 tokens for a prototype POAP backend.
/// @dev SECURITY: Uses Ownable and AccessControl for Role-Based Access Control (RBAC).
contract AttendanceBadges is ERC1155, ERC1155Supply, Ownable, AccessControl, ReentrancyGuard {
  /// @notice Role allowed to issue existing POAP badges.
  bytes32 public constant ISSUER_ROLE = keccak256("ISSUER_ROLE");

  /// @notice First token ID assigned by this contract.
  uint256 public constant FIRST_TOKEN_ID = 1;

  /// @notice Maximum number of token IDs returned by holder enumeration helpers.
  uint256 public constant MAX_TRACKED_TOKENS_PER_QUERY = 100;

  /// @notice Packed event configuration and minting counters.
  /// @dev Four uint64 values fit into one storage slot; bool is kept separate for clarity.
  struct POAPEvent {
    uint64 startTime;
    uint64 endTime;
    uint64 maxSupply;
    uint64 minted;
    bool active;
  }

  uint256 private _nextTokenId = FIRST_TOKEN_ID;
  uint256[] private _createdTokenIds;
  uint256 private _totalIssued;
  uint256 private _uniqueHoldersCount;

  mapping(uint256 tokenId => POAPEvent poapEvent) private _poapEvents;
  mapping(uint256 tokenId => string tokenUri) private _tokenUris;
  mapping(uint256 tokenId => bool created) private _created;
  mapping(address holder => uint256[] tokenIds) private _holderTokenIds;
  mapping(address holder => mapping(uint256 tokenId => bool tracked)) private _holderTokenTracked;
  mapping(address holder => bool isHolder) private _isHolder;

  /// @notice Emitted when an owner creates a new POAP badge type.
  /// @param tokenId The ERC-1155 token ID for the badge type.
  /// @param maxSupply Maximum number of badges that may be issued.
  /// @param startTime Event start timestamp.
  /// @param endTime Event end timestamp.
  /// @param tokenUri Metadata URI for the badge type.
  event POAPEventCreated(uint256 indexed tokenId, uint64 maxSupply, uint64 startTime, uint64 endTime, string tokenUri);

  /// @notice Emitted when a POAP badge is issued to an attendee.
  /// @param tokenId The ERC-1155 token ID issued.
  /// @param attendee Recipient wallet address.
  /// @param issuer Account that issued the badge.
  event POAPIssued(uint256 indexed tokenId, address indexed attendee, address indexed issuer);

  /// @notice Emitted when a POAP badge type is enabled or disabled.
  /// @param tokenId The ERC-1155 token ID updated.
  /// @param active Whether the badge type can currently be issued.
  event POAPEventStatusChanged(uint256 indexed tokenId, bool active);

  /// @dev SECURITY: RBAC check centralizes owner-or-issuer authorization for minting.
  modifier onlyIssuer() {
    require(owner() == _msgSender() || hasRole(ISSUER_ROLE, _msgSender()), "POAP: caller is not issuer");
    _;
  }

  /// @notice Initializes the ERC-1155 POAP contract.
  /// @param initialOwner Owner/admin address for privileged operations.
  /// @param defaultUri Fallback ERC-1155 metadata URI.
  constructor(address initialOwner, string memory defaultUri) ERC1155(defaultUri) Ownable(initialOwner) {
    require(initialOwner != address(0), "POAP: owner is zero address");
    require(bytes(defaultUri).length > 0, "POAP: default URI is empty");

```

```
poap-events on  main [$!] is 📦 v1.0.0 via ⬢ v24.15.0 via ❄  impure
❯ npx hardhat compile

Compiled 1 Solidity file with solc 0.8.28 (evm target: cancun)
```

```
poap-events on  main [$!] is 📦 v1.0.0 via ⬢ v24.15.0 via ❄  impure
❯ npx hardhat compile
No contracts to compile

poap-events on  main [$!] is 📦 v1.0.0 via ⬢ v24.15.0 via ❄  impure
❯ npx hardhat test
No contracts to compile

Running Solidity tests


Running Mocha tests


  AttendanceBadges
    deployment
      ✔ initializes owner, issuer role, ERC-1155 interface, and next token ID
      ✔ rejects invalid constructor arguments
    event creation
      ✔ creates a badge type and exposes its metadata
      ✔ rejects invalid event creation input
      ✔ restricts event creation and status changes to the owner
    issuing badges
      ✔ allows the owner to issue a POAP and updates attendee reads
      ✔ allows an owner-granted issuer to issue badges
      ✔ batch issues one POAP to multiple attendees
      ✔ rejects invalid issue attempts
      ✔ enforces max supply for single and batch issue flows
      ✔ rejects invalid batch issue input
    admin roles and read validation
      ✔ grants and revokes issuer permissions
      ✔ rejects invalid role and read inputs
      ✔ emits status changes and rejects missing token status updates
    analytics
      ✔ tracks total issued POAPs and unique holders globally


  15 passing (229ms)


15 passing (15 mocha)
```

Gas Usage Statistics

contracts/AttendanceBadges.sol:AttendanceBadges

Function name	Min	Average	Median	Max	#calls
balanceOf	24249	24249	24249	24249	2
balanceOfBatch	33096	33096	33096	33096	1
batchIssuePOAPs	493923	493923	493923	493923	1
createPOAPEvent	149984	168101	167228	194949	12
DEFAULT_ADMIN_ROLE	21291	21291	21291	21291	1
getCreatedTokenIds	26113	26113	26113	26113	1
getHolderPOAPBalances	29831	29831	29831	29831	1
getHolderTokenIds	26913	26913	26913	26913	1
getPOAPEvent	28554	28554	28554	28554	4
grantIssuer	48691	48691	48691	48691	2
hasPOAP	26527	26527	26527	26527	1
hasRole	24354	24642	24738	24738	4
isCreatedToken	23784	23784	23784	23784	1
issuePOAP	152112	223011	245837	248169	7
ISSUER_ROLE	21343	21343	21343	21343	2
nextTokenId	23390	23390	23390	23390	14
owner	23453	23453	23453	23453	1
revokeIssuer	26651	26651	26651	26651	1
setPOAPEventActive	27803	27803	27803	27803	2
supportsInterface	21905	21905	21905	21905	1
totalIssued	23476	23476	23476	23476	4
uniqueHoldersCount	23411	23411	23411	23411	4
uri	26738	26738	26738	26738	1

Deployment	Min	Average	Median	Max	#deployments
	2596918	2596918	2596918	2596918	15
Bytecode size	10818				

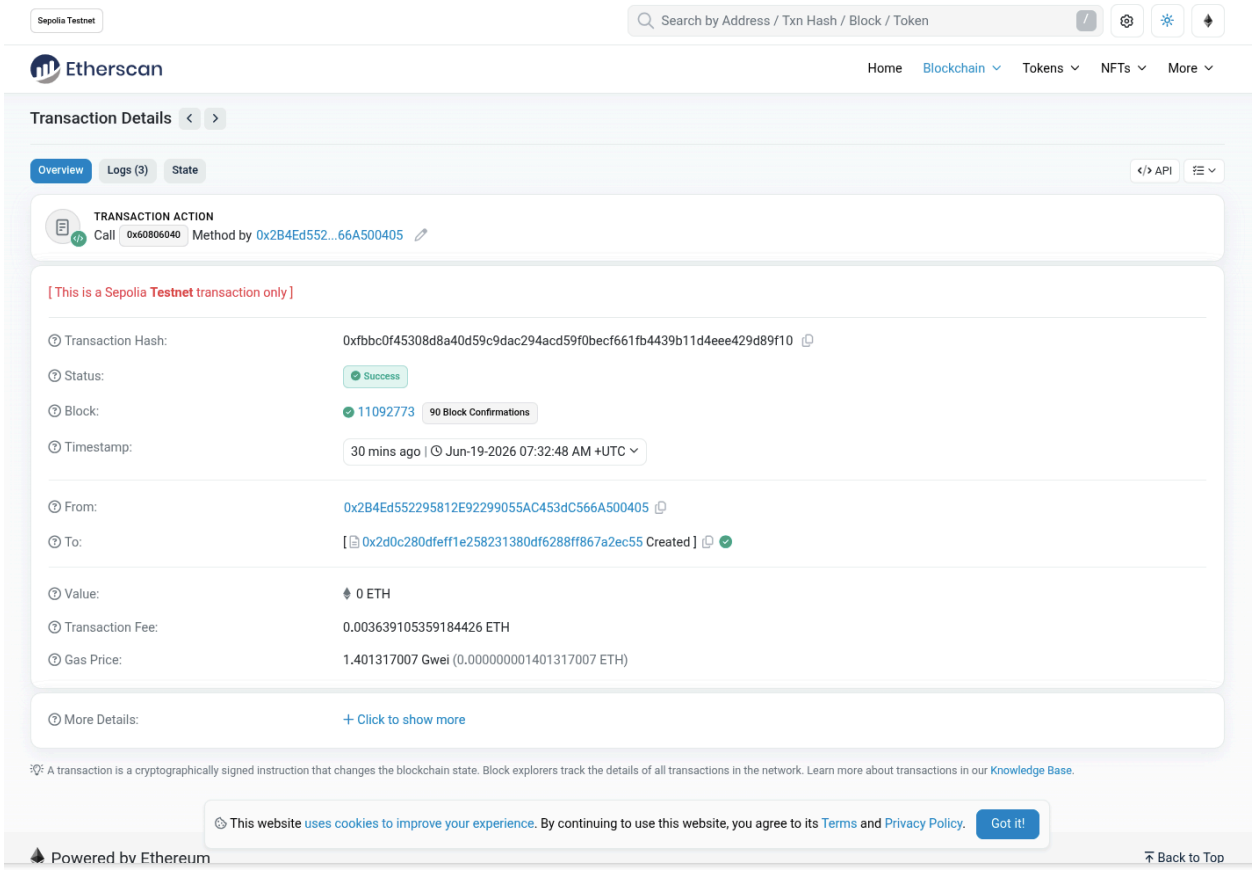
Coverage Report

File Coverage

File Path	Line %	Statement %	Uncovered Lines
contracts/AttendanceBadges.sol	100.00	100.00	-
Total	100.00	100.00	

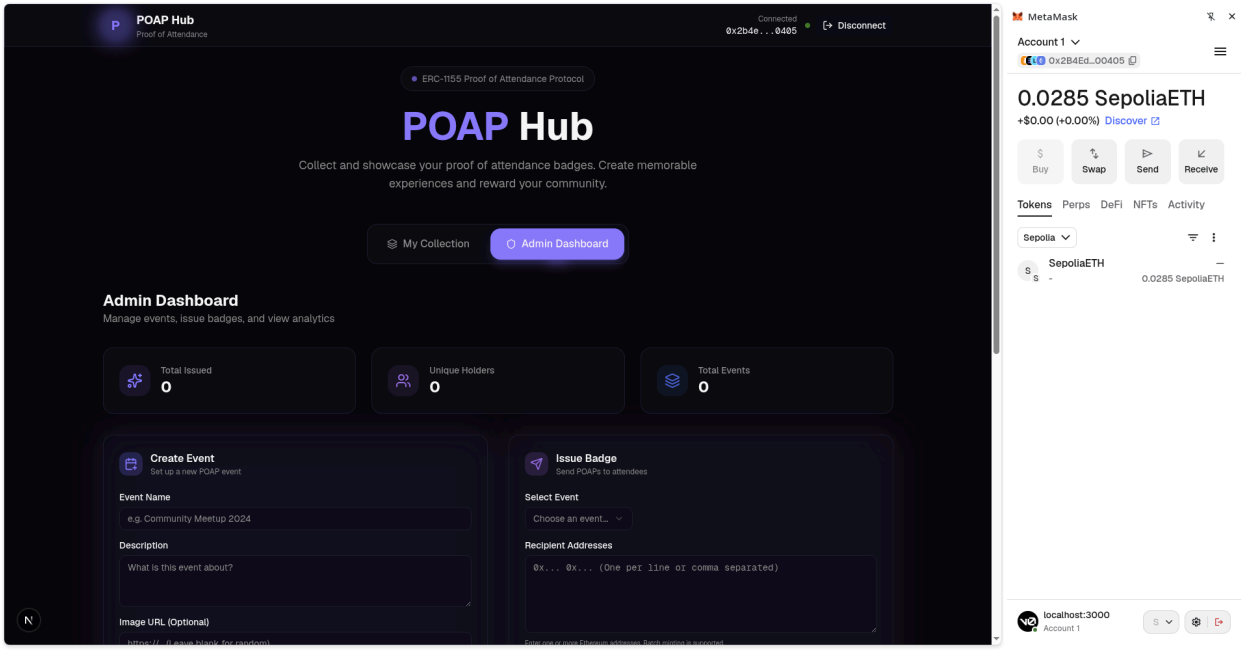
File: deployments/sepolia/AttendanceBadges.json

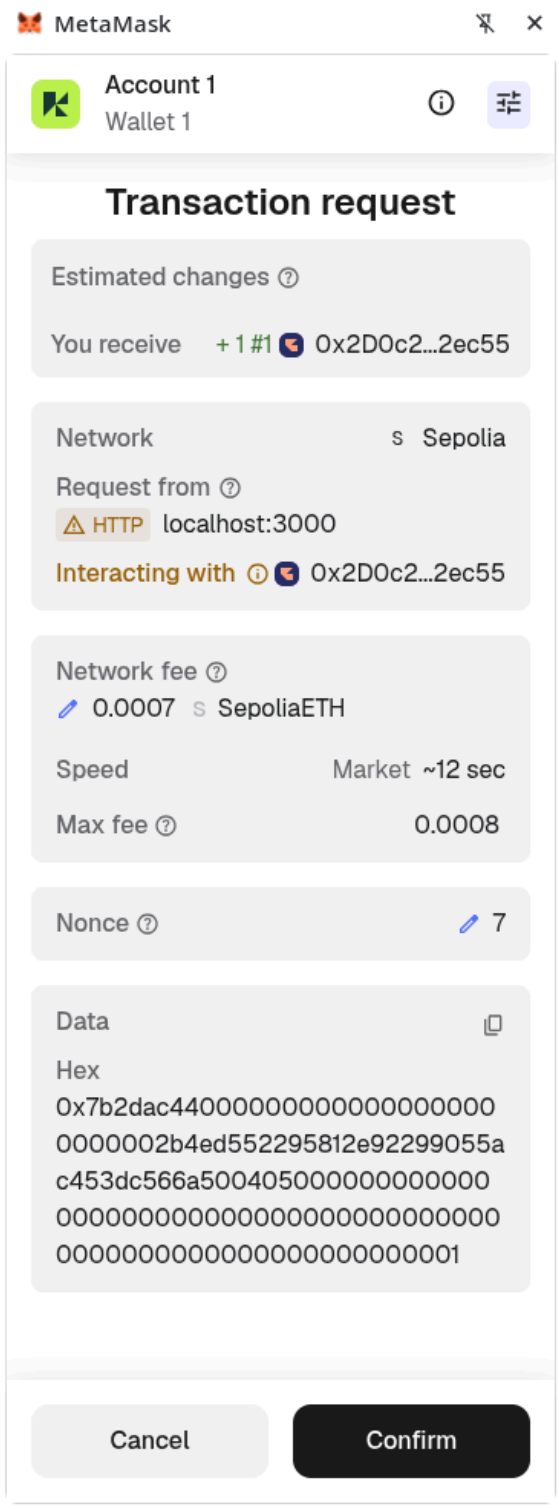

```
{
  "contractName": "AttendanceBadges",
  "network": "sepolia",
  "chainId": "11155111",
  "contractAddress": "0xb4342aB9c0035FED4a3b25e9455e2106C4D97E24",
  "deployer": "0xa016462001e464C1979683783b638Ebcf73A1d94",
  "transactionHash": "0xf4e215f3c87a65d4dfb2c6a7810a1a6bb8dbe2702d4c2fb7baf46c5efc059b1b",
  "blockNumber": 10933285,
  "deployedAt": "2026-05-27T12:36:50.943Z",
  "constructorArguments": [
    "0xa016462001e464C1979683783b638Ebcf73A1d94",
    "https://example.com/metadata/"
  ],
  "explorerUrl": "https://sepolia.etherscan.io/address/0xb4342aB9c0035FED4a3b25e9455e2106C4D97E24",
  "abi": [
    {
      "inputs": [
        {
          "internalType": "address",
          "name": "initialOwner",
          "type": "address"
        },
        {
          "internalType": "string",
          "name": "defaultUri",
          "type": "string"
        }
      ],
      "stateMutability": "nonpayable",
      "type": "constructor"
    },
    {
      "inputs": [],
      "name": "AccessControlBadConfirmation",
      "type": "error"
    },
    {
      "inputs": [
        {
          "internalType": "address",
          "name": "account",
          "type": "address"
        },
        {
          "internalType": "bytes32",
          "name": "neededRole",
          "type": "bytes32"
        }
      ],
      "name": "AccessControlUnauthorizedAccount",
      "type": "error"
    },
    {
```

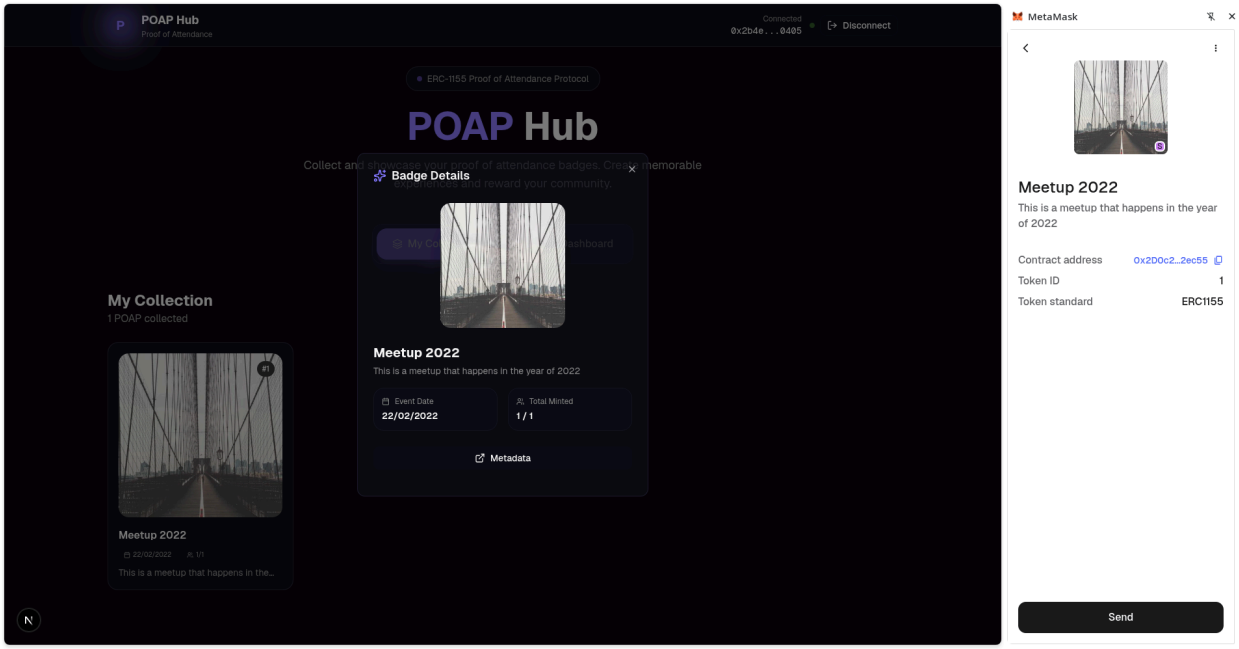

Sepolia Testnet
Search by Address / Txn Hash / Block / Token
Etherscan
Home
Blockchain
Tokens
NFTs
More
Transaction Details
Overview
Logs (3)
State
API
TRANSACTION ACTION
Call 0x60806040 Method by 0x2B4Ed552...66A500405
[This is a Sepolia Testnet transaction only]
Transaction Hash:
0xfbbc0f45308d8a40d59c9dac294acd59f0becf661fb4439b11d4eee429d89f10
Status:
Success
Block:
11092773
90 Block Confirmations
Timestamp:
30 mins ago | Jun-19-2026 07:32:48 AM +UTC
From:
0x2B4Ed552295812E92299055AC453dC566A500405
To:
[0x2d0c280dfeff1e258231380df6288ff867a2ec55 Created]
Value:
0 ETH
Transaction Fee:
0.003639105359184426 ETH
Gas Price:
1.401317007 Gwei (0.000000001401317007 ETH)
More Details:
+ Click to show more
A transaction is a cryptographically signed instruction that changes the blockchain state. Block explorers track the details of all transactions in the network. Learn more about transactions in our Knowledge Base.
This website uses cookies to improve your experience. By continuing to use this website, you agree to its Terms and Privacy Policy.
Got it!
Powered by Ethereum
Back to Top

File: deployments/sepolia/AttendanceBadges.abi.json


```
[
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "initialOwner",
        "type": "address"
      },
      {
        "internalType": "string",
        "name": "defaultUri",
        "type": "string"
      }
    ],
    "stateMutability": "nonpayable",
    "type": "constructor"
  },
  {
    "inputs": [],
    "name": "AccessControlBadConfirmation",
    "type": "error"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "account",
        "type": "address"
      },
      {
        "internalType": "bytes32",
        "name": "neededRole",
        "type": "bytes32"
      }
    ],
    "name": "AccessControlUnauthorizedAccount",
    "type": "error"
  },
  {
    "inputs": [
      {
        "internalType": "address",
        "name": "sender",
        "type": "address"
      },
      {
        "internalType": "uint256",
        "name": "balance",
        "type": "uint256"
      },
      {
        "internalType": "uint256",
        "name": "needed",
```


POAP Hub
Proof of Attendance
Connected
0x2b4e...0405
Disconnect
ERC-1155 Proof of Attendance Protocol
POAP Hub
Collect and showcase your proof of attendance badges. Create memorable experiences and reward your community.
My Collection
Admin Dashboard
Admin Dashboard
Manage events, issue badges, and view analytics
Total Issued
0
Unique Holders
0
Total Events
0
Create Event
Set up a new POAP event
Event Name
e.g. Community Meetup 2024
Description
What is this event about?
Image URL (Optional)
Issue Badge
Send POAPs to attendees
Select Event
Choose an event...
Recipient Addresses
0x... 0x... (One per line or comma separated)
MetaMask
Account 1
0x2B4Ed...00405
0.0285 SepoliaETH
+$0.00 (+0.00%) Discover
Buy
Swap
Send
Receive
Tokens
Perps
DeFi
NFTs
Activity
Sepolia
SepoliaETH
0.0285 SepoliaETH
localhost:3000
Account 1


MetaMask
Account 1
Wallet 1
Transaction request
Estimated changes
You receive
+ 1 #1
0x2D0c2...2ec55
Network
Sepolia
Request from
HTTP
localhost:3000
Interacting with
0x2D0c2...2ec55
Network fee
0.0007
SepoliaETH
Speed
Market
~12 sec
Max fee
0.0008
Nonce
7
Data
Hex
0x7b2dac44000000000000000000000002b4ed552295812e92299055ac453dc566a500405000000000000000000000000000000000000000000000000000000000000001
Cancel
Confirm


POAP Hub
Proof of Attendance
Connected
0x2b4e...0405
Disconnect
ERC-1155 Proof of Attendance Protocol
POAP Hub
Collect and showcase your proof of attendance badges. Create memorable experiences and reward your community.
My Collection
1 POAP collected
#1
Meetup 2022
22/02/2022
1/1
This is a meetup that happens in the...
Badge Details
Meetup 2022
This is a meetup that happens in the year of 2022
Event Date
22/02/2022
Total Minted
1 / 1
Metadata
MetaMask
Meetup 2022
This is a meetup that happens in the year of 2022
Contract address
0x2D0c2...2ec55
Token ID
1
Token standard
ERC1155
Send